

Федеральное государственное автономное образовательное учреждение
высшего профессионального образования
«Московский государственный технический университет имени Н.Э. Баумана»
(МГТУ им. Н.Э. Баумана)
Мытищинский филиал

КУРСОВАЯ РАБОТА
ПО ДИСЦИПЛИНЕ: «БАЗЫ ДАННЫХ»
НА ТЕМУ: «ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ
БАЗЫ ДАННЫХ ДЛЯ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ
ПЕРСОНАЛИЗИРОВАННОГО ПИТАНИЯ NUTRIAGENT»

Выполнил: Костин А.С. КЗ-84Б

Проверил: Брюквина О.Ю.

Мытищи, 2026

Содержание

Введение	3
1. Описание предметной области.....	3
1.1 Общее описание предметной области	3
1.2 Описание входных документов.....	4
1.3 Список ограничений.....	4
2. Проектирование реляционной базы данных	5
2.1 Инфологическая модель базы данных.....	5
2.1.1 Описание сущностей	6
2.1.2 Построение с помощью теоремы о нормализации.....	6
2.1.3 Описание связей	7
2.1.4 Диаграмма связи по полям	8
2.2 Даталогическая модель базы данных	8
3. Организация выборки информации из базы данных	9
3.1 Простые запросы	9
3.2 Сложные запросы	11
4. Разработка технологий доступа к базе данных	13
4.1 Выбор пользователей базы данных	13
4.2 Разграничение полномочий пользователя	13
Заключение.....	15
Список литературы.....	15
Приложение А. SQL-скрипт создания базы данных.....	16
Приложение Б. Триггеры базы данных	19
Приложение В. Роли и права доступа	20

Введение

Информационные системы и базы данных являются основой современных сервисов, работающих с персональными данными, рекомендациями и аналитикой. В сфере питания особенно важно не только хранить список рецептов, но и связывать его с индивидуальными параметрами пользователя: возрастом, массой тела, ростом, уровнем активности, целью, аллергиями, нежелательными ингредиентами и медицинскими ограничениями.

Предметом данной курсовой работы является проектирование и реализация реляционной базы данных для интеллектуальной системы NutriAgent. Система предназначена для формирования персонализированных планов питания, расчета калорийности и КБЖУ, хранения каталога рецептов, аудита генераций и подготовки аналитических отчетов.

В качестве СУБД выбрана PostgreSQL. Для хранения векторных представлений рецептов используется расширение pgvector, а Redis применяется только как вспомогательный слой: очередь задач, live-progress и краткосрочный кэш ответов. Долговременным источником истины остается PostgreSQL, что позволяет обеспечить ссылочную целостность, нормализацию данных, транзакционность, контроль ограничений и возможность формирования SQL-отчетности.

Цель работы - спроектировать и описать базу данных NutriAgent, обеспечивающую полный цикл: регистрацию пользователя, хранение профиля питания, ведение нормализованного каталога рецептов, генерацию недельного плана питания, сохранение состава блюд, фиксацию событий изменения плана и аудит работы генератора.

1. Описание предметной области

1.1 Общее описание предметной области

Предметной областью является работа интеллектуальной системы персонализированного питания. Пользователь проходит анкетирование, указывает физиологические параметры и ограничения, после чего система подбирает блюда из проверенного каталога и формирует план питания на несколько дней. Для каждого блюда сохраняются калории, белки, жиры, углеводы и список ингредиентов.

NutriAgent объединяет несколько информационных процессов: учет пользователей, учет ограничений питания, хранение рецептов, проверку новых источников рецептов, генерацию плана, расчет дневных итогов,

хранение истории генераций и подготовку отчетов для анализа качества системы.

- Учет пользователей: email, хэш пароля, возраст, рост, вес, пол, активность, цель и рассчитанная целевая калорийность.
- Учет ограничений: аллергены, предпочтения, нежелательные ингредиенты, заболевания и индивидуальное расписание приемов пищи.
- Каталог рецептов: название, описание, КБЖУ, тип приема пищи, категория, время приготовления, ингредиенты, теги, аллергены и embedding для подбора.
- План питания: шапка плана, дни, приемы пищи, snapshot ингредиентов и событийная история изменений.
- Контроль генераций: запись task id, режима, статуса, качества, модели, версии prompt и детальных шагов выполнения.
- Каталог-кандидаты: недоверенные источники и рецепты проходят staging и review перед попаданием в каноническую таблицу recipes.

Особое внимание уделяется надежности. Рецепты и планы не хранятся как один JSON-документ: состав рецепта, строки плана, приемы пищи и ингредиенты вынесены в отдельные таблицы. Это позволяет выполнять аналитические запросы, строить список покупок, проверять ограничения пользователя и не терять историю даже при очистке Redis.

1.2 Описание входных документов

Для построения базы данных были проанализированы следующие входные данные и документы, используемые в системе NutriAgent:

- Анкета пользователя: возраст, рост, вес, пол, активность, цель питания, аллергии, предпочтения, нежелательные ингредиенты и заболевания.
- Каталог рецептов: исходные JSON-файлы, результаты ingestion, нормализованные ингредиенты, теги и аллергены.
- Результаты генерации плана: дни, приемы пищи, время приема, порции, КБЖУ, ингредиенты и список покупок.
- Журналы выполнения: task id, mode, status, quality status, ошибки и шаги генерации.
- Кандидаты источников: URL, домен, тип источника, snapshot, provenance, validation report и статус проверки.

1.3 Список ограничений

- Каждая таблица имеет первичный ключ id типа uuid.
- Email пользователя уникален и обязателен для заполнения.

- Калории, белки, жиры и углеводы не могут быть отрицательными.
- Порядок ингредиента, порядок приема пищи и номер дня должны быть положительными.
- Процент калорий meal_schedule_slots.calories_pct находится в диапазоне от 1 до 100.
- План питания не может иметь end_date раньше start_date.
- Списки аллергий, предпочтений, заболеваний и нежелательных ингредиентов не дублируются в рамках одного пользователя.
- При удалении пользователя каскадно удаляются его ограничения, расписание и генерации.
- При удалении рецепта из готового плана ссылка на recipe_id сбрасывается, но title_snapshot и КБЖУ блюда сохраняются для истории.
- Redis не считается долговременным источником истины; готовый план должен восстанавливаться из PostgreSQL.

2. Проектирование реляционной базы данных

2.1 Инфологическая модель базы данных

На инфологическом этапе выделяются сущности предметной области и связи между ними без привязки к физической реализации. Для NutriAgent центральными сущностями являются Users, Recipes, Meal_Plans, Generation_Runs и Catalog Candidates. Вокруг них расположены нормализованные таблицы, раскрывающие состав профиля, рецепта и готового плана питания.

Инфологическая модель БД NutriAgent

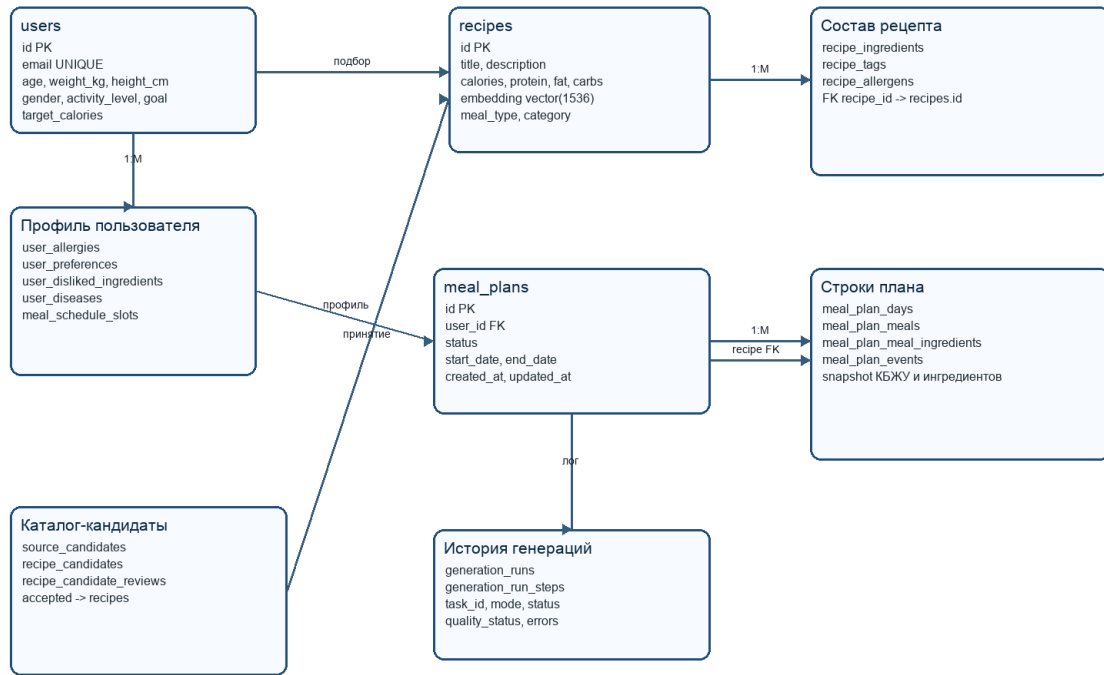


Рисунок 1. Инфологическая модель базы данных NutriAgent

2.1.1 Описание сущностей

Сущность	Назначение
Users	Хранит учетную запись и базовые поля анкеты: email, password_hash, age, weight_kg, height_cm, gender, activity_level, goal, target_calories.
User profile tables	Хранят нормализованные списки: аллергии, предпочтения, нежелательные ингредиенты, заболевания и расписание приемов пищи.
Recipes	Канонический каталог блюд с КБЖУ, описанием, типом приема пищи, категорией, временем приготовления и embedding vector(1536).
Recipe details	Ингредиенты, теги и аллергии рецепта. Таблицы отделены от recipes для соблюдения 1НФ и аналитических запросов.
Meal Plans	Шапка плана питания: пользователь, статус, дата начала, дата окончания и служебные timestamps.
Meal Plan rows	Дни плана, блюда, ингредиенты в составе блюда и события изменения плана.
Generation Runs	История работы генератора: task id, режим, статус, качество, модель, версия prompt, ошибка и шаги.
Catalog Staging	Недоверенные source_candidates, recipe_candidates и recipe_candidate_reviews перед попаданием в рецепты.

2.1.2 Построение с помощью теоремы о нормализации отношений

База данных приведена к третьей нормальной форме. Первая нормальная форма выполняется, так как повторяющиеся списки профиля и рецепта вынесены из JSON/ARRAY-полей в отдельные строки таблиц.

Например, аллергии пользователя хранятся в `user_allergies`, а ингредиенты рецепта - в `recipe_ingredients`.

Вторая нормальная форма выполняется, так как каждая таблица имеет собственный первичный ключ, а все неключевые атрибуты зависят от него целиком. Например, в `meal_plan_meals` значения `calories`, `protein`, `fat`, `carbs` и `title_snapshot` описывают конкретную строку приема пищи, а не внешний план целиком.

Третья нормальная форма выполняется, так как транзитивные зависимости вынесены в отдельные отношения. Категоризация рецептов, аллергии, теги, ингредиенты и ограничения пользователя не дублируются внутри шапок плана или пользователя. Это снижает избыточность и уменьшает риск аномалий обновления.

2.1.3 Описание связей

N	Родительская таблица	Дочерняя таблица	Тип	Поле связи	Смысл
1	users	user_allergies	1:M	user_id	Аллергены пользователя
2	users	user_preferences	1:M	user_id	Предпочтения пользователя
3	users	user_disliked_ingredients	1:M	user_id	Нежелательные ингредиенты
4	users	user_diseases	1:M	user_id	Ограничения по заболеваниям
5	users	meal_schedule_slots	1:M	user_id	Расписание приемов пищи
6	recipes	recipe_ingredients	1:M	recipe_id	Состав рецепта
7	recipes	recipe_tags	1:M	recipe_id	Теги рецепта
8	recipes	recipe_allergens	1:M	recipe_id	Аллергены рецепта
9	users	meal_plans	1:M	user_id	Планы пользователя
10	meal_plans	meal_plan_days	1:M	meal_plan_id	Дни плана
11	meal_plan_days	meal_plan_meals	1:M	day_id	Приемы пищи
12	meal_plan_meals	meal_plan_meal_ingredients	1:M	meal_id	Snapshot ингредиентов
13	recipes	meal_plan_meals	1:M	recipe_id	Использование рецепта
14	meal_plans	meal_plan_events	1:M	meal_plan_id	Аудит изменений
15	generation_runs	generation_run_steps	1:M	generation_run_id	Шаги генерации
16	recipe_candidates	recipe_candidate_reviews	1:M	candidate_id	Проверки кандидата

2.1.4 Диаграмма связи по полям

Поток данных при генерации персонализированного плана питания

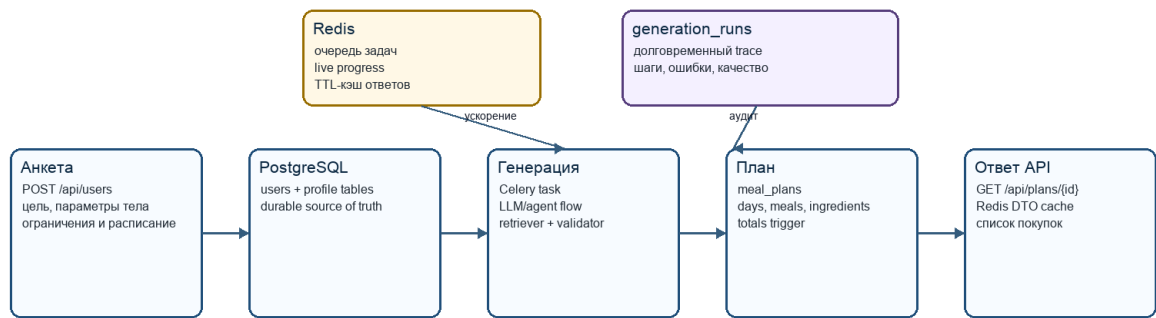


Рисунок 2. Поток данных при генерации персонализированного плана питания

2.2 Даталогическая модель базы данных

Даталогическая модель описывает структуру реляционной базы данных в терминах таблиц, полей, типов данных, первичных и внешних ключей. Для идентификаторов используется `uuid`, для временных данных - `date`, `time` и `timestamp`, для числовых показателей питания - `double precision`, для `embedding` рецептов - `vector(1536)`.

Основной состав таблиц БД:

- `users(id, email, password_hash, age, weight_kg, height_cm, gender, activity_level, goal, target_calories, created_at, updated_at)`;
- `user_allergies(id, user_id, allergen)`, `user_preferences(id, user_id, preference)`, `user_disliked_ingredients(id, user_id, ingredient)`, `user_diseases(id, user_id, disease)`;
- `meal_schedule_slots(id, user_id, slot_order, meal_type, planned_time, calories_pct)`;
- `recipes(id, title, description, calories, protein, fat, carbs, embedding, meal_type, ingredients_short, prep_time_min, category)`;
- `recipe_ingredients(id, recipe_id, position, name, amount, unit)`, `recipe_tags(id, recipe_id, tag)`, `recipe_allergens(id, recipe_id, allergen)`;
- `meal_plans(id, user_id, status, start_date, end_date, created_at, updated_at)`;

- meal_plan_days(id, meal_plan_id, day_number, plan_date, total_calories, total_protein, total_fat, total_carbs);
- meal_plan_meals(id, day_id, recipe_id, meal_type, planned_time, title_snapshot, calories, protein, fat, carbs, portion_factor);
- meal_plan_meal_ingredients(id, meal_id, position, ingredient_name, amount, unit);
- meal_plan_events(id, meal_plan_id, event_type, actor_type, old_meal_id, new_meal_id, reason, created_at);
- generation_runs(id, user_id, meal_plan_id, task_id, mode, status, quality_status, model_name, prompt_version, error_message, started_at, finished_at);
- generation_run_steps(id, generation_run_id, step_key, status, message, started_at, finished_at);
- source_candidates, recipe_candidates, recipe_candidate_reviews для workflow наполнения каталога.

Для повышения надежности используются внешние ключи, ЧЕК-ограничения, уникальные ограничения и индексы по часто используемым полям: user_id, recipe_id, allergen, tag, task_id, status и domain. Итоговые значения КБЖУ по дню пересчитываются триггером при изменении строк meal_plan_meals.

3. Организация выборки информации из базы данных

Для проверки практической применимости базы данных подготовлен набор SQL-запросов. Простые запросы решают типовые операционные задачи: поиск рецептов, просмотр планов, получение ингредиентов и выявление задач с ошибками. Сложные запросы формируют аналитические отчеты по качеству планов, использованию рецептов, ограничениям пользователей и эффективности генерации.

3.1 Простые запросы

1. Пользователи с целью снижения веса

```
SELECT id, email, target_calories
FROM users
WHERE goal = 'lose';
```

id	email	target_calories
...91a	demo@nutriagent.ru	1840

2. Рецепты до 500 ккал

```
SELECT id, title, calories
FROM recipes
WHERE calories < 500
```

```
ORDER BY calories;
```

title	calories
Овсянка с ягодами	360
Куриный салат	430

3. Рецепты для завтрака

```
SELECT id, title, calories
FROM recipes
WHERE meal_type = 'breakfast';
```

title	meal_type
Творожная тарелка	breakfast

4. Поиск рецептов с аллергеном

```
SELECT r.id, r.title, a.allergen
FROM recipes r
JOIN recipe_allergens a ON a.recipe_id = r.id
WHERE a.allergen = 'milk';
```

title	allergen
Творожная тарелка	milk

5. Состав одного рецепта

```
SELECT position, name, amount, unit
FROM recipe_ingredients
WHERE recipe_id = :recipe_id
ORDER BY position;
```

position	name	amount	unit
1	ОВСЯНЫЕ ХЛОПЬЯ	60	g

6. Планы питания пользователя

```
SELECT id, status, start_date, end_date
FROM meal_plans
WHERE user_id = :user_id
ORDER BY created_at DESC;
```

status	start_date	end_date
READY	2026-05-04	2026-05-10

7. Дни одного плана

```
SELECT day_number, plan_date, total_calories, total_protein, total_fat, total_carbs
FROM meal_plan_days
WHERE meal_plan_id = :meal_plan_id
ORDER BY day_number;
```

day_number	total_calories	total_protein
1	1845	132

8. Приемы пищи в одном дне

```
SELECT meal_type, planned_time, title_snapshot, calories
FROM meal_plan_meals
WHERE day_id = :day_id
```

```
ORDER BY planned_time;
```

meal type	planned time	title snapshot
breakfast	08:00	Овсянка с ягодами

9. Кандидаты рецептов на проверке

```
SELECT id, source_url, source_type, created_at
FROM recipe_candidates
WHERE status = 'REVIEW'
ORDER BY created_at DESC;
```

source type	created at
web	2026-05-01 12:10

10. Неуспешные генерации

```
SELECT id, task_id, mode, status, started_at
FROM generation_runs
WHERE status = 'FAILED'
ORDER BY started_at DESC;
```

task_id	mode	status
celery-...	api	FAILED

3.2 Сложные запросы

1. Средняя калорийность планов по целям пользователей

```
SELECT u.goal, AVG(d.total_calories) AS avg_daily_calories
FROM users u
JOIN meal_plans p ON p.user_id = u.id
JOIN meal_plan_days d ON d.meal_plan_id = p.id
WHERE p.status = 'READY'
GROUP BY u.goal
ORDER BY u.goal;
```

2. ТОП используемых рецептов в готовых планах

```
SELECT r.id, r.title, COUNT(*) AS used_count
FROM meal_plan_meals m
JOIN recipes r ON r.id = m.recipe_id
GROUP BY r.id, r.title
ORDER BY used_count DESC, r.title
LIMIT 10;
```

3. Отклонение дневной калорийности от цели пользователя

```
SELECT u.email, p.id AS meal_plan_id, d.day_number, u.target_calories,
       d.total_calories, d.total_calories - u.target_calories AS deviation_kcal
FROM users u
JOIN meal_plans p ON p.user_id = u.id
JOIN meal_plan_days d ON d.meal_plan_id = p.id
WHERE u.target_calories IS NOT NULL
ORDER BY ABS(d.total_calories - u.target_calories) DESC;
```

4. Полный отчет КБЖУ по одному плану

```

SELECT d.day_number, m.meal_type, m.planned_time, m.title_snapshot,
       m.calories, m.protein, m.fat, m.carbs
FROM meal_plan_days d
JOIN meal_plan_meals m ON m.day_id = d.id
WHERE d.meal_plan_id = :meal_plan_id
ORDER BY d.day_number, m.planned_time;

```

5. Процент принятия источников каталога

```

SELECT sc.domain, COUNT(*) AS total_sources,
       COUNT(*) FILTER (WHERE sc.status = 'ACCEPTED') AS accepted_sources,
       ROUND(COUNT(*) FILTER (WHERE sc.status = 'ACCEPTED')::numeric /
NULLIF(COUNT(*), 0) * 100, 2) AS accepted_pct
FROM source_candidates sc
GROUP BY sc.domain
ORDER BY accepted_pct DESC NULLS LAST, total_sources DESC;

```

6. Средняя длительность генераций по режимам

```

SELECT mode, COUNT(*) AS runs, AVG(finished_at - started_at) AS avg_duration
FROM generation_runs
WHERE finished_at IS NOT NULL
GROUP BY mode
ORDER BY avg_duration DESC;

```

7. Пользователи с максимальным числом ограничений

```

SELECT u.email,
       COUNT(DISTINCT ua.allergen) AS allergies_count,
       COUNT(DISTINCT ud.disease) AS diseases_count,
       COUNT(DISTINCT di.ingredient) AS disliked_count
FROM users u
LEFT JOIN user_allergies ua ON ua.user_id = u.id
LEFT JOIN user_diseases ud ON ud.user_id = u.id
LEFT JOIN user_disliked_ingredients di ON di.user_id = u.id
GROUP BY u.id, u.email
ORDER BY (COUNT(DISTINCT ua.allergen) + COUNT(DISTINCT ud.disease) + COUNT(DISTINCT
di.ingredient)) DESC;

```

8. Покрытие каталога по типам приемов пищи

```

SELECT meal_type, COUNT(*) AS recipes_count, AVG(calories) AS avg_calories,
       MIN(calories) AS min_calories, MAX(calories) AS max_calories
FROM recipes
GROUP BY meal_type
ORDER BY recipes_count DESC;

```

9. Наиболее редактируемые планы

```

SELECT p.id AS meal_plan_id, u.email, COUNT(e.id) AS events_count,
       MAX(e.created_at) AS last_event_at
FROM meal_plans p
JOIN users u ON u.id = p.user_id
LEFT JOIN meal_plan_events e ON e.meal_plan_id = p.id
GROUP BY p.id, u.email

```

```
ORDER BY events_count DESC, last_event_at DESC NULLS LAST;
```

10. Список покупок для одного плана

```
SELECT LOWER(i.ingredient_name) AS ingredient, i.unit, SUM(i.amount) AS
total_amount
FROM meal_plan_days d
JOIN meal_plan_meals m ON m.day_id = d.id
JOIN meal_plan_meal_ingredients i ON i.meal_id = m.id
WHERE d.meal_plan_id = :meal_plan_id
GROUP BY LOWER(i.ingredient_name), i.unit
ORDER BY ingredient;
```

goal	avg daily calories	recipes count	avg duration	accepted_pct
lose	1847.3	42	00:00:18	78.50
maintain	2210.6	35	00:00:21	81.25

Таблица 2. Пример агрегированного результата аналитических запросов

4. Разработка технологий доступа к базе данных

4.1 Выбор пользователей базы данных

Для разграничения полномочий в NutriAgent выделены четыре роли PostgreSQL. Такое разделение позволяет отделить администрирование схемы, работу backend-приложения, работу специалиста по питанию и режим только для чтения.

Роль	Пользователь	Права	Назначение
nutri_admin	Администратор БД	Полный доступ к таблицам и последовательностям, управление схемой, миграциями, резервным копированием и восстановлением.	Минимизация риска несанкционированного изменения данных
nutri_app	Backend-приложение	Операционная запись пользователей, планов, генераций и событий; чтение канонического каталога рецептов.	Минимизация риска несанкционированного изменения данных
nutri_nutritionist	Специалист по питанию	Проверка и пополнение каталога: рецепты, ингредиенты, теги, аллергены, кандидаты и review.	Минимизация риска несанкционированного изменения данных
nutri_readonly	Аналитик	Чтение всех таблиц без права изменения данных.	Минимизация риска несанкционированного изменения данных

4.2 Разграничение полномочий пользователя

Разграничение прав выполняется средствами PostgreSQL через CREATE ROLE и GRANT. Приложение получает права только на те таблицы, которые необходимы для выполнения пользовательских сценариев:

регистрация, генерация плана, сохранение строк плана, фиксация событий и чтение рецептов.

Специалист по питанию может изменять каталог рецептов и таблицы workflow-кандидатов, но не должен иметь прямой доступ к паролям пользователей или служебным данным генераций. Аналитическая роль получает только SELECT, что позволяет строить отчеты без риска повреждения производственной базы.

```
-- PostgreSQL role sketch for NutriAgent.
-- Adjust passwords/secrets outside version control.

CREATE ROLE nutri_admin;
CREATE ROLE nutri_app;
CREATE ROLE nutri_nutritionist;
CREATE ROLE nutri_readonly;

GRANT CONNECT ON DATABASE nutriagent TO nutri_admin, nutri_app, nutri_nutritionist,
nutri_readonly;
GRANT USAGE ON SCHEMA public TO nutri_admin, nutri_app, nutri_nutritionist,
nutri_readonly;

GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO nutri_admin;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO nutri_admin;

GRANT SELECT, INSERT, UPDATE, DELETE ON
    users,
    user_allergies,
    user_preferences,
    user_disliked_ingredients,
    user_diseases,
    meal_schedule_slots,
    meal_plans,
    meal_plan_days,
    meal_plan_meals,
    meal_plan_meal_ingredients,
    meal_plan_events,
    generation_runs,
    generation_run_steps
TO nutri_app;

GRANT SELECT ON
    recipes,
    recipe_ingredients,
    recipe_tags,
    recipe_allergens
TO nutri_app;

GRANT SELECT, INSERT, UPDATE ON
    recipes,
    recipe_ingredients,
    recipe_tags,
    recipe_allergens,
    recipe_candidates,
    recipe_candidate_reviews,
```

```
source_candidates  
TO nutri_nutritionist;
```

```
GRANT SELECT ON ALL TABLES IN SCHEMA public TO nutri_readonly;
```

Заключение

В ходе выполнения курсовой работы была спроектирована и описана реляционная база данных для интеллектуальной системы персонализированного питания NutriAgent. На основе анализа предметной области выделены основные сущности: пользователи, ограничения питания, рецепты, ингредиенты, планы питания, события плана, генерации и workflow наполнения каталога.

Разработанная структура ориентирована на PostgreSQL и приведена к третьей нормальной форме. Повторяющиеся списки и вложенные документы вынесены в отдельные таблицы, что позволяет выполнять надежные SQL-запросы, строить аналитические отчеты, проверять ограничения пользователя и формировать список покупок по готовому плану. Для ускорения работы используется Redis, однако долговременным источником истины остается PostgreSQL.

Практическая значимость базы данных подтверждается набором простых и сложных запросов, системой ролей доступа и использованием триггеров для автоматического обновления timestamps, пересчета дневных итогов КБЖУ и аудита изменений плана. Такая архитектура может использоваться как основа для полнофункционального сервиса персонализированного питания с контролируемой генерацией и проверяемыми данными.

Список литературы

1. Документация PostgreSQL. <https://www.postgresql.org/docs/>
2. Документация pgvector. <https://github.com/pgvector/pgvector>
3. Date C. J. An Introduction to Database Systems. 8th ed. Pearson, 2003.
4. Connolly T., Begg C. Database Systems: A Practical Approach to Design, Implementation and Management. 6th ed. Pearson, 2014.
5. Метанит. PostgreSQL и SQL. <https://metanit.com/sql/postgresql/>
6. Документация SQLAlchemy. <https://docs.sqlalchemy.org/>
7. Документация Alembic. <https://alembic.sqlalchemy.org/>
8. Документация FastAPI. <https://fastapi.tiangolo.com/>

9. Материалы проекта NutriAgent: docs/db/schema.sql, docs/db/triggers.sql, docs/db/roles.sql.

10. Ревунков Г. И. и др. Методические указания к курсовой работе по дисциплине «Базы данных». МГТУ им. Н. Э. Баумана, 2019.

Приложение А. SQL-скрипт создания базы данных

```
-- NutriAgent normalized PostgreSQL schema overview.
-- Executable migrations live in backend/alembic/versions.

CREATE TABLE users (
    id uuid PRIMARY KEY,
    email varchar(255) NOT NULL UNIQUE,
    password_hash varchar(255) NOT NULL,
    age integer NOT NULL,
    weight_kg double precision NOT NULL,
    height_cm double precision NOT NULL,
    gender varchar NOT NULL,
    activity_level varchar NOT NULL,
    goal varchar NOT NULL,
    target_calories integer,
    created_at timestamp,
    updated_at timestamp NOT NULL DEFAULT now()
);

CREATE TABLE user_allergies (
    id uuid PRIMARY KEY,
    user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    allergen varchar(100) NOT NULL,
    UNIQUE (user_id, allergen)
);

CREATE TABLE user_preferences (
    id uuid PRIMARY KEY,
    user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    preference varchar(100) NOT NULL,
    UNIQUE (user_id, preference)
);

CREATE TABLE user_disliked_ingredients (
    id uuid PRIMARY KEY,
    user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    ingredient varchar(100) NOT NULL,
    UNIQUE (user_id, ingredient)
);

CREATE TABLE user_diseases (
    id uuid PRIMARY KEY,
    user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    disease varchar(100) NOT NULL,
    UNIQUE (user_id, disease)
);

CREATE TABLE meal_schedule_slots (
    id uuid PRIMARY KEY,
    user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
```

```

        slot_order integer NOT NULL CHECK (slot_order >= 1),
        meal_type varchar(50) NOT NULL,
        planned_time time NOT NULL,
        calories_pct integer NOT NULL CHECK (calories_pct > 0 AND calories_pct <= 100),
        UNIQUE (user_id, meal_type, planned_time)
    );

```

```

CREATE TABLE recipes (
    id uuid PRIMARY KEY,
    title varchar(255) NOT NULL,
    description text,
    calories double precision NOT NULL CHECK (calories >= 0),
    protein double precision NOT NULL CHECK (protein >= 0),
    fat double precision NOT NULL CHECK (fat >= 0),
    carbs double precision NOT NULL CHECK (carbs >= 0),
    embedding vector(1536),
    meal_type varchar(50),
    ingredients_short varchar(500),
    prep_time_min integer,
    category varchar(100),
    created_at timestamp,
    updated_at timestamp NOT NULL DEFAULT now()
);

```

```

CREATE TABLE recipe_ingredients (
    id uuid PRIMARY KEY,
    recipe_id uuid NOT NULL REFERENCES recipes(id) ON DELETE CASCADE,
    position integer NOT NULL CHECK (position >= 1),
    name varchar(255) NOT NULL,
    amount double precision NOT NULL CHECK (amount >= 0),
    unit varchar(50) NOT NULL,
    UNIQUE (recipe_id, position)
);

```

```

CREATE TABLE recipe_tags (
    id uuid PRIMARY KEY,
    recipe_id uuid NOT NULL REFERENCES recipes(id) ON DELETE CASCADE,
    tag varchar(100) NOT NULL,
    UNIQUE (recipe_id, tag)
);

```

```

CREATE TABLE recipe_allergens (
    id uuid PRIMARY KEY,
    recipe_id uuid NOT NULL REFERENCES recipes(id) ON DELETE CASCADE,
    allergen varchar(100) NOT NULL,
    UNIQUE (recipe_id, allergen)
);

```

```

CREATE TABLE meal_plans (
    id uuid PRIMARY KEY,
    user_id uuid NOT NULL REFERENCES users(id),
    status varchar NOT NULL,
    start_date date,
    end_date date,
    created_at timestamp,
    updated_at timestamp NOT NULL DEFAULT now(),
    CHECK (end_date IS NULL OR start_date IS NULL OR end_date >= start_date)
);

```

```

CREATE TABLE meal_plan_days (
    id uuid PRIMARY KEY,
    meal_plan_id uuid NOT NULL REFERENCES meal_plans(id) ON DELETE CASCADE,
    day_number integer NOT NULL CHECK (day_number >= 1),
    plan_date date,
    total_calories double precision NOT NULL DEFAULT 0 CHECK (total_calories >= 0),
    total_protein double precision NOT NULL DEFAULT 0 CHECK (total_protein >= 0),
    total_fat double precision NOT NULL DEFAULT 0 CHECK (total_fat >= 0),
    total_carbs double precision NOT NULL DEFAULT 0 CHECK (total_carbs >= 0),
    UNIQUE (meal_plan_id, day_number)
);

```

```

CREATE TABLE meal_plan_meals (
    id uuid PRIMARY KEY,
    day_id uuid NOT NULL REFERENCES meal_plan_days(id) ON DELETE CASCADE,
    recipe_id uuid REFERENCES recipes(id) ON DELETE SET NULL,
    meal_type varchar(50) NOT NULL,
    planned_time time,
    title_snapshot varchar(255) NOT NULL,
    calories double precision NOT NULL CHECK (calories >= 0),
    protein double precision NOT NULL CHECK (protein >= 0),
    fat double precision NOT NULL CHECK (fat >= 0),
    carbs double precision NOT NULL CHECK (carbs >= 0),
    portion_factor double precision NOT NULL DEFAULT 1 CHECK (portion_factor > 0)
);

```

```

CREATE TABLE meal_plan_meal_ingredients (
    id uuid PRIMARY KEY,
    meal_id uuid NOT NULL REFERENCES meal_plan_meals(id) ON DELETE CASCADE,
    position integer NOT NULL CHECK (position >= 1),
    ingredient_name varchar(255) NOT NULL,
    amount double precision NOT NULL CHECK (amount >= 0),
    unit varchar(50) NOT NULL,
    UNIQUE (meal_id, position)
);

```

```

CREATE TABLE meal_plan_events (
    id uuid PRIMARY KEY,
    meal_plan_id uuid NOT NULL REFERENCES meal_plans(id) ON DELETE CASCADE,
    event_type varchar(50) NOT NULL,
    actor_type varchar(50) NOT NULL DEFAULT 'system',
    old_meal_id uuid REFERENCES meal_plan_meals(id) ON DELETE SET NULL,
    new_meal_id uuid REFERENCES meal_plan_meals(id) ON DELETE SET NULL,
    reason text,
    created_at timestamp NOT NULL DEFAULT now()
);

```

```

CREATE TABLE generation_runs (
    id uuid PRIMARY KEY,
    user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    meal_plan_id uuid REFERENCES meal_plans(id) ON DELETE SET NULL,
    task_id varchar(255),
    mode varchar(50) NOT NULL,
    status varchar(50) NOT NULL,
    quality_status varchar(50),
    model_name varchar(255),
    prompt_version varchar(100),
    error_message text,

```

```

        started_at timestamp NOT NULL DEFAULT now(),
        finished_at timestamp
    );

CREATE TABLE generation_run_steps (
    id uuid PRIMARY KEY,
    generation_run_id uuid NOT NULL REFERENCES generation_runs(id) ON DELETE
    CASCADE,
    step_key varchar(100) NOT NULL,
    status varchar(50) NOT NULL,
    message text,
    started_at timestamp NOT NULL DEFAULT now(),
    finished_at timestamp
);

```

Приложение Б. Триггеры базы данных

```
-- Trigger functions mirrored by Alembic migrations.
```

```

CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS trigger AS $$
BEGIN
    NEW.updated_at = now();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE OR REPLACE FUNCTION recalculate_meal_plan_day_totals()
RETURNS trigger AS $$
DECLARE
    target_day_id uuid;
BEGIN
    target_day_id = COALESCE(NEW.day_id, OLD.day_id);

    UPDATE meal_plan_days
    SET
        total_calories = COALESCE((SELECT SUM(calories) FROM meal_plan_meals WHERE
day_id = target_day_id), 0),
        total_protein = COALESCE((SELECT SUM(protein) FROM meal_plan_meals WHERE
day_id = target_day_id), 0),
        total_fat = COALESCE((SELECT SUM(fat) FROM meal_plan_meals WHERE day_id =
target_day_id), 0),
        total_carbs = COALESCE((SELECT SUM(carbs) FROM meal_plan_meals WHERE day_id
= target_day_id), 0),
        updated_at = now()
    WHERE id = target_day_id;

    RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_meal_plan_meals_recalculate_day_totals
AFTER INSERT OR UPDATE OR DELETE ON meal_plan_meals
FOR EACH ROW EXECUTE FUNCTION recalculate_meal_plan_day_totals();

```

```

CREATE OR REPLACE FUNCTION audit_meal_plan_update()

```

```

RETURNS trigger AS $$
BEGIN
    IF OLD.status IS DISTINCT FROM NEW.status
    OR OLD.start_date IS DISTINCT FROM NEW.start_date
    OR OLD.end_date IS DISTINCT FROM NEW.end_date THEN
        INSERT INTO meal_plan_events (id, meal_plan_id, event_type, actor_type,
reason)
            VALUES (gen_random_uuid(), NEW.id, 'meal_plan_updated', 'database_trigger',
CONCAT('status=', OLD.status, '->', NEW.status));
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_meal_plans_audit_update
AFTER UPDATE ON meal_plans
FOR EACH ROW EXECUTE FUNCTION audit_meal_plan_update();

```

Приложение В. Роли и права доступа

```

-- PostgreSQL role sketch for NutriAgent.
-- Adjust passwords/secrets outside version control.

CREATE ROLE nutri_admin;
CREATE ROLE nutri_app;
CREATE ROLE nutri_nutritionist;
CREATE ROLE nutri_readonly;

GRANT CONNECT ON DATABASE nutriagent TO nutri_admin, nutri_app, nutri_nutritionist,
nutri_readonly;
GRANT USAGE ON SCHEMA public TO nutri_admin, nutri_app, nutri_nutritionist,
nutri_readonly;

GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO nutri_admin;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO nutri_admin;

GRANT SELECT, INSERT, UPDATE, DELETE ON
    users,
    user_allergies,
    user_preferences,
    user_disliked_ingredients,
    user_diseases,
    meal_schedule_slots,
    meal_plans,
    meal_plan_days,
    meal_plan_meals,
    meal_plan_meal_ingredients,
    meal_plan_events,
    generation_runs,
    generation_run_steps
TO nutri_app;

GRANT SELECT ON
    recipes,
    recipe_ingredients,
    recipe_tags,
    recipe_allergens

```

```
TO nutri_app;
```

```
GRANT SELECT, INSERT, UPDATE ON  
    recipes,  
    recipe_ingredients,  
    recipe_tags,  
    recipe_allergens,  
    recipe_candidates,  
    recipe_candidate_reviews,  
    source_candidates  
TO nutri_nutritionist;
```

```
GRANT SELECT ON ALL TABLES IN SCHEMA public TO nutri_readonly;
```